MIRS: Monitor Inteligente de Recursos del Sistema

Entrega 2: levantamiento de información, descripción y estructura del prototipo

Leidy Yohanna Cardona Bayona y Gustavo Adolfo Cardona Bayona

Facultad de Ingeniería, Universidad Manuela Beltrán

Sistemas Operativos, Grupo 2

Ing. Javier Ochoa

4 de noviembre de 2026

Tabla de contenido

1. Introducción....4

2. Levantamiento de información....5

2.1 Usuarios involucrados....5

2.2 Necesidad....5

2.3 Problemática....6

2.4 Pregunta problema....7

3. Descripción del proyecto....8

3.1 Objetivos....8

3.2 Requerimientos funcionales....9

3.3 Requerimientos no funcionales....9

3.4 Detección de anomalías....10

3.5 Alcance....10

4. Estructura del prototipo....12

4.1 Arquitectura general....12

4.2 Frontend: interfaz de usuario....14

4.3 Control de versiones y commits....17

4.4 Base de datos....19

4.5 Modelo relacional....21

4.6 Lo que queda definido y lo que sigue....23

5. Tecnologías del prototipo....24

6. Conclusiones....26

Referencias....27

1. Introducción

Todo sistema operativo reparte entre los procesos activos el tiempo del procesador, la memoria principal y el acceso a los dispositivos de almacenamiento (Silberschatz et al., 2018). En un equipo de uso diario ese reparto pasa desapercibido hasta que algún programa consume más de lo normal: la máquina se vuelve lenta y quien la usa no sabe qué proceso lo provoca ni desde cuándo ocurre. MIRS, el Monitor Inteligente de Recursos del Sistema, es el proyecto que el grupo 2 desarrolla durante la asignatura para atender esa situación con una aplicación de escritorio que vigila los recursos, compara el consumo con el historial del propio equipo y avisa a tiempo.

Este documento corresponde a la segunda entrega y continúa la ficha técnica presentada en la primera. Su propósito es dejar definida la base sobre la cual se construirá el prototipo. El apartado 2 recoge el levantamiento de información, con la necesidad y la problemática; el 3 describe el proyecto, sus requerimientos y su alcance; el 4 presenta la estructura del prototipo en cuatro frentes (interfaz o frontend, control de versiones con los commits del repositorio, base de datos y modelo relacional), y el 5 justifica las tecnologías elegidas.

2. Levantamiento de información

El levantamiento se apoyó en tres fuentes: el uso que los integrantes del grupo hacen de sus equipos de estudio y de trabajo, la revisión de las herramientas de supervisión que ya incluye Windows y la bibliografía de la asignatura sobre procesos y gestión de memoria. A partir de ellas se identificaron los usuarios del sistema, sus necesidades y el problema que el proyecto busca resolver.

2.1 Usuarios involucrados

El usuario principal es la persona que estudia o trabaja en un computador con Windows y que no tiene formación en administración de sistemas: un estudiante, un empleado de una oficina pequeña o un trabajador independiente. Hay un segundo perfil, el de quien presta soporte técnico en una oficina sin área de sistemas, que necesita saber qué ocurrió en un equipo cuando alguien reporta lentitud. Los integrantes del grupo y el docente participan como evaluadores del prototipo, no como usuarios finales.

2.2 Necesidad

Lo que estos usuarios piden no es más información técnica, sino información interpretada y oportuna. Quieren saber qué programa está afectando el equipo en el momento en que lo notan, recibir un aviso sin tener que vigilar la pantalla y poder revisar después lo que pasó. La Tabla 1 resume las necesidades encontradas y la forma en que MIRS responde a cada una.

Tabla 1

Necesidades identificadas y respuesta del proyecto

Código	Necesidad	Usuario	Cómo la atiende MIRS
N1	Saber qué proceso consume los recursos cuando el equipo se pone lento	Usuario del equipo	Panel en vivo y lista de procesos ordenada por consumo (RF-01, RF-02)
N2	Recibir un aviso sin tener que estar pendiente del equipo	Usuario del equipo	Alertas automáticas con severidad y recomendación (RF-04, RF-05)

Código	Necesidad	Usuario	Cómo la atiende MIRS
N3	Distinguir un pico pasajero de un consumo sostenido o de un crecimiento anormal de memoria	Usuario y soporte	Umbral con duración y comparación con el historial (RF-03, RF-04)
N4	Conocer lo que ocurrió en el equipo en las horas anteriores	Soporte técnico	Historial en base de datos, gráficas e informe (RF-07)
N5	Actuar sobre el proceso problemático sin cerrar algo por error	Usuario del equipo	Acciones con confirmación previa y registro (RF-06)

Nota. Los códigos RF remiten a los requerimientos funcionales de la Tabla 3.

2.3 Problemática

Windows ofrece herramientas para revisar el rendimiento. El Administrador de tareas muestra el consumo de cada proceso en el instante en que se abre. El Monitor de rendimiento va más lejos: grafica contadores en tiempo real con un intervalo predeterminado de un segundo, permite programar conjuntos de recopiladores de datos y puede generar alertas cuando un contador alcanza un umbral (Microsoft, s.f.). Sin embargo, este segundo programa está pensado para administradores; es un complemento de la consola de administración, sus ejemplos se orientan a servidores y su uso exige elegir contadores, crear los conjuntos y programarlos. En la práctica, el usuario común solo tiene a la mano el Administrador de tareas, que no conserva historial y sirve únicamente si se consulta justo cuando ocurre el problema.

De ahí resulta un ciclo que se repite. El equipo se vuelve lento, el usuario no identifica la causa, espera o reinicia, y el problema reaparece más tarde sin que nadie sepa qué lo originó. Los casos más difíciles de notar son los graduales: un programa que reserva memoria y no la libera reduce poco a poco la memoria disponible, el sistema recurre cada vez más a la paginación en disco y el rendimiento cae sin un evento visible que lo explique (Silberschatz et al., 2018). La Tabla 2 relaciona cada causa con su consecuencia.

Tabla 2

Síntesis de la problemática

Causa	Consecuencia
La herramienta disponible para el usuario común solo muestra el estado actual	La causa de la lentitud se pierde si no se observa en el momento exacto
Las herramientas con historial y alertas requieren conocimientos de administración	El usuario no las configura y queda sin registro de lo ocurrido
No existe una referencia del comportamiento normal del equipo	No se distingue un pico pasajero de un problema persistente
El crecimiento gradual de memoria no produce un evento visible	El rendimiento se degrada hasta que el usuario reinicia sin conocer la causa

El problema tiene además un valor académico. Los conceptos de proceso, estado, prioridad y memoria que se estudian en teoría (Stallings, 2018) se vuelven observables cuando una aplicación los lee del sistema operativo y los muestra en su evolución a lo largo del tiempo.

2.4 Pregunta problema

¿Cómo puede una aplicación de escritorio que lee del sistema operativo el estado de los procesos y de los recursos ayudar a un usuario sin formación técnica a identificar a tiempo qué afecta el rendimiento de su equipo y a actuar sobre esa causa?

3. Descripción del proyecto

MIRS es una aplicación de escritorio para Windows, escrita en Python, que toma una muestra del estado del equipo cada uno o dos segundos, la guarda en una base de datos local y la compara con el historial para decidir si hay una anomalía. Cuando la encuentra, presenta una alerta con su severidad, el proceso involucrado y una recomendación. El usuario puede revisar el comportamiento de las últimas horas, ajustar los umbrales y, si lo decide, finalizar un proceso o cambiar su prioridad.

3.1 Objetivos

Objetivo general. Desarrollar una aplicación de escritorio que monitoree en tiempo real el uso de CPU, memoria, disco y procesos de un equipo con Windows, detecte consumos anómalos mediante umbrales y análisis estadístico del historial, y genere alertas y recomendaciones para el usuario.

Objetivos específicos.

1. Identificar los mecanismos del sistema operativo que exponen el estado de los procesos y de los recursos, y la forma de leerlos desde Python.
2. Diseñar la arquitectura del prototipo, su interfaz y el modelo de datos del historial.
3. Implementar la recolección periódica de métricas, el almacenamiento y la detección de anomalías.
4. Construir la interfaz gráfica con el panel en vivo, la lista de procesos, las alertas y el historial.
5. Validar el prototipo con pruebas automáticas y con escenarios controlados de alto consumo de CPU y de memoria.

Esta entrega cumple los objetivos 1 y 2 y adelanta parte del 3: el recolector de métricas y el almacenamiento ya funcionan y tienen pruebas.

3.2 Requerimientos funcionales

Tabla 3

Requerimientos funcionales y estado en la Entrega 2

Código	Requerimiento	Estado en esta entrega
RF-01	Mostrar en tiempo real el uso de CPU, RAM, disco y red	Recolección implementada; pantalla diseñada
RF-02	Listar los procesos activos con PID, nombre, estado, prioridad, CPU, memoria e hilos	Recolección implementada; pantalla diseñada
RF-03	Configurar un umbral y una duración por recurso	Tabla y valores iniciales implementados
RF-04	Detectar anomalías por umbral sostenido, desviación del historial y crecimiento de memoria	Reglas definidas (sección 3.4)
RF-05	Generar alertas con severidad y recomendación	Tabla alerta definida; pantalla diseñada
RF-06	Finalizar un proceso o cambiar su prioridad con confirmación y registro	Tabla accion definida; diálogo diseñado
RF-07	Guardar el historial y presentarlo en gráficas y en un informe	Almacenamiento implementado; gráficas diseñadas

3.3 Requerimientos no funcionales

Se definieron cinco requerimientos no funcionales. Cada uno tiene una métrica y un procedimiento de prueba, de modo que su cumplimiento pueda verificarse y no quede como una intención.

Tabla 4

Requerimientos no funcionales con métrica y prueba

Código	Requerimiento	Métrica	Procedimiento de prueba
RNF-01	Actualización periódica	Entre 1 y 2 s entre muestras consecutivas	Registrar 100 muestras y calcular la diferencia entre marcas de tiempo
RNF-02	Bajo consumo propio	Promedio del proceso de MIRS menor a 5 % de CPU y a 150 MB de memoria	Medir el propio proceso con psutil durante 30 minutos de uso normal
RNF-03	Integridad del historial	Ninguna muestra se guarda sin sus procesos ni con porcentajes fuera de 0 a 100	Pruebas unitarias de transacción y de restricciones CHECK (implementadas)

Código	Requerimiento	Métrica	Procedimiento de prueba
RNF-04	Seguridad de las acciones	El 100 % de las acciones sobre procesos pide confirmación y queda registrado	Cancelar y confirmar acciones desde la interfaz y revisar la tabla accion
RNF-05	Compatibilidad	Funciona en Windows 10 y 11 con Python 3.10 o superior	Ejecutar la suite de pruebas en ambos sistemas

3.4 Detección de anomalías

En la detección de anomalías se concentra lo que el nombre del proyecto llama inteligente. Chandola et al. (2009) describen la detección de anomalías como la búsqueda de patrones que no se ajustan al comportamiento esperado y llaman anomalía puntual al dato individual que se aparta del resto. MIRS aplica esa idea con estadística descriptiva y no con modelos entrenados; así cada alerta puede explicarse al usuario con cifras del propio equipo y el desarrollo cabe dentro del curso. La Tabla 5 muestra las tres reglas previstas.

Tabla 5

Reglas de detección de anomalías

Regla	Condición	Severidad	Ejemplo de alerta
Umbral sostenido	El recurso supera valor_limite durante duracion_seg (tabla umbral)	ALTA	CPU sobre 85 % durante 60 s
Desviación del historial	El valor actual se aleja más de tres desviaciones estándar del promedio móvil de las últimas muestras	MEDIA	Uso de RAM inusual para este equipo
Posible fuga de memoria	La memoria de un mismo proceso crece en muestras consecutivas durante una ventana configurable, sin bajar	MEDIA	python.exe pasó de 210 a 612 MB en 15 min

Nota. El factor de tres desviaciones estándar y el tamaño de las ventanas son valores iniciales que se ajustarán con las pruebas de la siguiente entrega.

3.5 Alcance

El proyecto incluye el monitoreo de un solo equipo local, la visualización en tiempo real, la detección por umbrales y por análisis estadístico simple, las alertas con recomendación, las acciones básicas sobre procesos con confirmación, el historial con gráficas e informe, y las pruebas con su manual de usuario. Quedan fuera el monitoreo remoto de varios equipos, los

modelos de aprendizaje automático entrenados, cualquier modificación del núcleo o de los controladores, las funciones de antivirus y las versiones web o móviles.

4. Estructura del prototipo

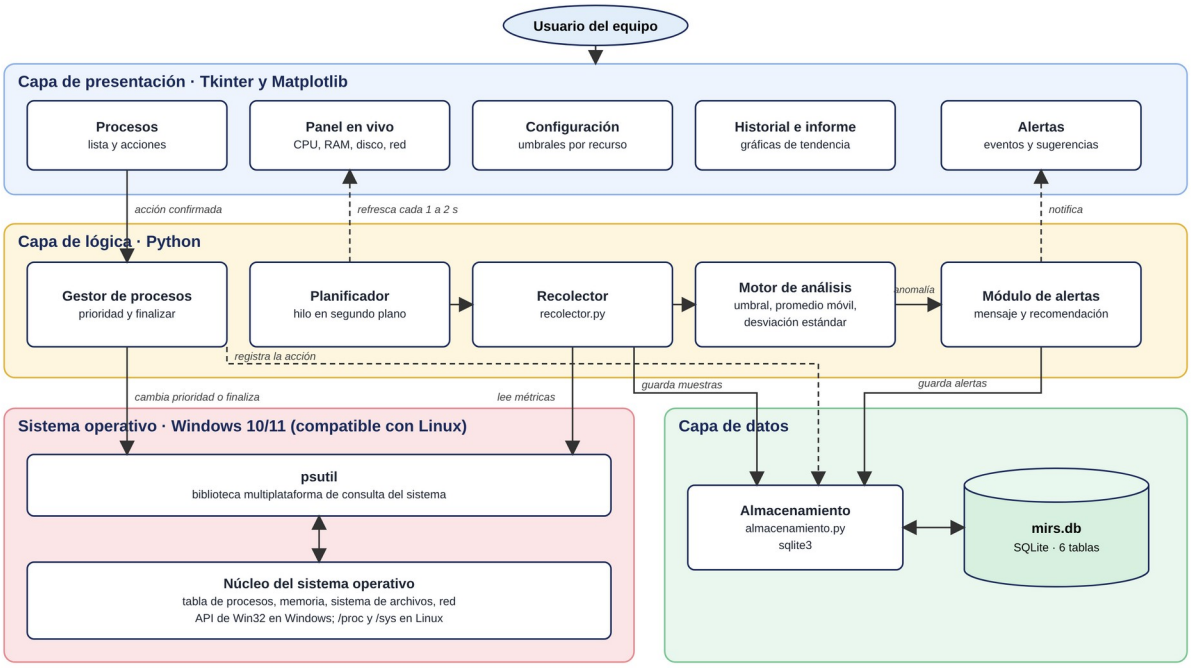
El prototipo se aloja en el repositorio https://github.com/gustavo8015/MIRS_SISTEMASOPERATIVOS_G2. En esta entrega contiene el recolector de métricas, el módulo de almacenamiento con su esquema SQLite, diez pruebas unitarias, los diagramas, los mockups de la interfaz y un README con la descripción contextual y técnica del proyecto. Los demás módulos ya están definidos en la arquitectura y se implementarán en las entregas siguientes.

4.1 Arquitectura general

La aplicación se organiza en tres capas que funcionan sobre el sistema operativo, como muestra la Figura 1. La capa de presentación reúne las cinco pantallas; la de lógica contiene el planificador de muestreo, el recolector, el motor de análisis, el módulo de alertas y el gestor de procesos, y la de datos guarda el historial en SQLite. La lectura del sistema se hace a través de psutil, una biblioteca multiplataforma que obtiene información de los procesos en ejecución y del uso de CPU, memoria, discos y red (Rodola, s.f.-a).

Figura 1

Arquitectura por capas de MIRS



Nota. Las flechas continuas indican llamadas y escritura de datos; las discontinuas, avisos hacia la interfaz o registro de acciones. Elaboración propia.

La interfaz no consulta al sistema operativo de forma directa. Tk trabaja con un único hilo y un ciclo de eventos, de manera que un cálculo largo dentro de un manejador congela la ventana; la documentación de Python recomienda dividir ese trabajo con temporizadores o llevarlo a otro hilo (Python Software Foundation, s.f.-b). Por esa razón el muestreo corre en un hilo en segundo plano que recolecta, analiza y guarda, mientras la interfaz se refresca cada uno o dos segundos con el método after() a partir de los datos ya almacenados. La Tabla 6 relaciona los módulos con sus archivos.

Tabla 6

Módulos del prototipo

Módulo	Archivo	Responsabilidad	Estado
Recolector de métricas	recolector.py	Leer CPU, RAM, disco, red y procesos con psutil	Implementado

Módulo	Archivo	Responsabilidad	Estado
Almacenamiento	almacenamiento.py	Crear el esquema y guardar equipo, umbrales y muestras	Implementado
Configuración	config.py	Intervalo, ruta de la base de datos y umbrales iniciales	Implementado
Planificador de muestreo	planificador.py	Hilo que coordina recolección, análisis y guardado	Entrega 3
Motor de análisis	analizador.py	Aplicar las reglas de la Tabla 5	Entrega 3
Módulo de alertas	alertas.py	Redactar el mensaje, la severidad y la recomendación	Entrega 3
Gestor de procesos	gestor_procesos.py	Cambiar prioridad o finalizar, con confirmación	Entrega 4
Interfaz gráfica	interfaz/	Pantallas Tkinter con gráficas de Matplotlib	Entrega 4

Nota. Todos los archivos están en src/mirs/ dentro del repositorio.

4.2 Frontend: interfaz de usuario

La interfaz se construirá con Tkinter y sus controles temáticos ttk, que vienen incluidos en Python, y con gráficas de Matplotlib insertadas en la ventana. Se organiza en cinco pestañas: Panel en vivo, Procesos, Alertas, Historial y Configuración. Para esta entrega se elaboraron mockups de las tres pantallas principales (Figuras 2 a 4); la Tabla 7 describe cada pestaña, los controles previstos y los requerimientos que atiende.

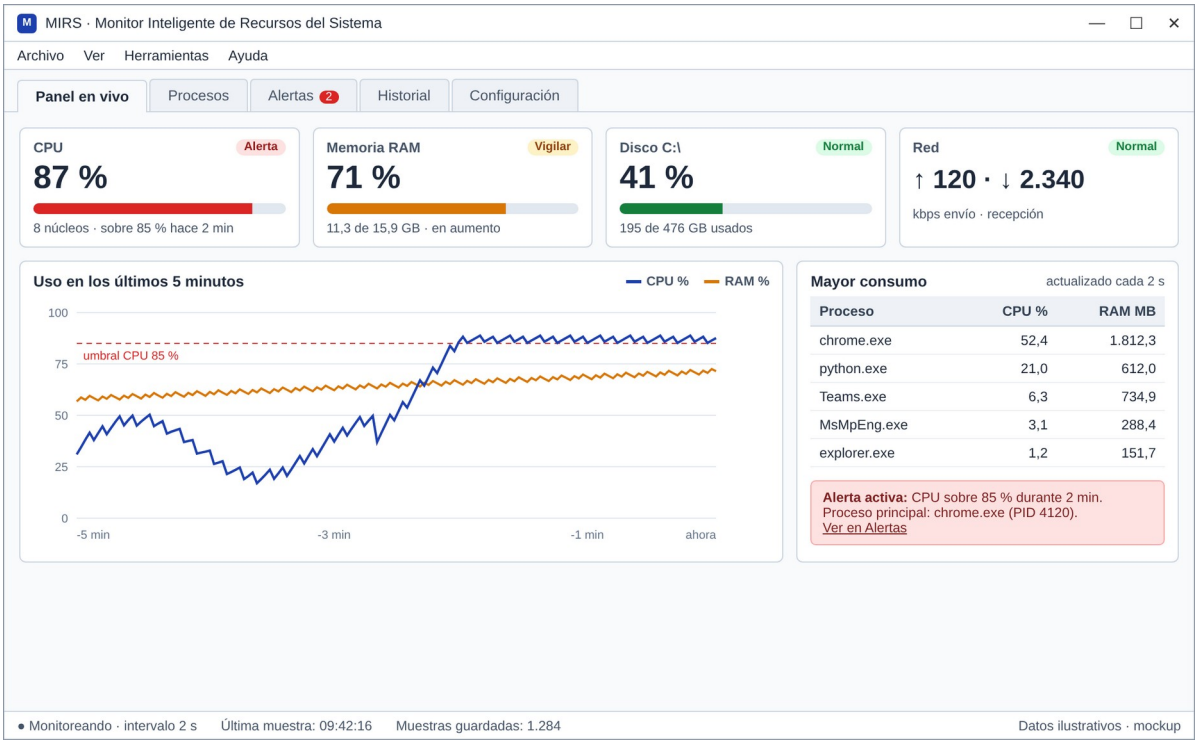
Tabla 7

Pantallas de la interfaz

Pestaña	Contenido	Controles Tkinter previstos	RF
Panel en vivo	Indicadores de CPU, RAM, disco y red; gráfica de los últimos 5 minutos; procesos de mayor consumo; alerta activa	ttk.Progressbar, ttk.Treeview, FigureCanvasTkAgg	RF-01, RF-07
Procesos	Tabla de procesos con búsqueda y orden; botones de acción; diálogo de confirmación	ttk.Treeview, ttk.Entry, ttk.Combobox, messagebox	RF-02, RF-06
Alertas	Alertas por severidad con mensaje y recomendación; ir al proceso; marcar como atendida o descartar	ttk.Treeview, ttk.Button	RF-04, RF-05
Historial	Gráficas por rango de fechas e informe resumen	FigureCanvasTkAgg, ttk.Button	RF-07
Configuración	Umbral y duración por recurso; intervalo de muestreo	ttk.Spinbox, ttk.Checkbutton	RF-03

Figura 2

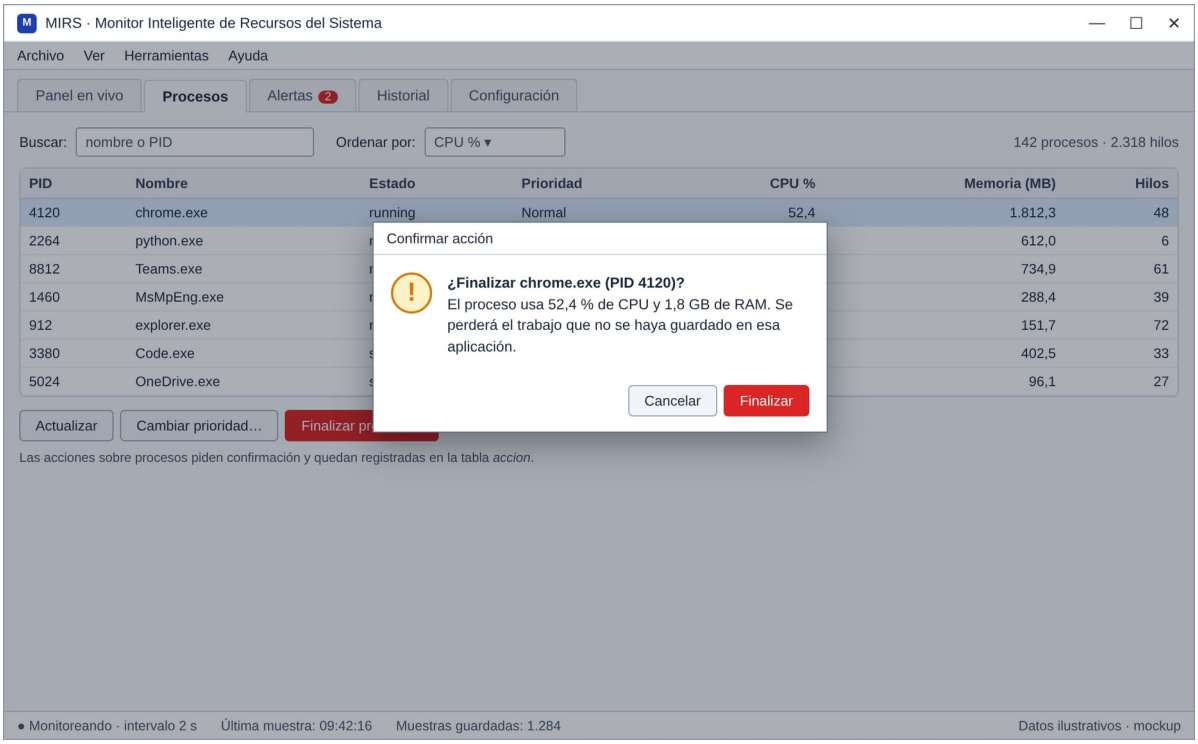
Mockup de la pestaña Panel en vivo



Nota. Mockup elaborado en HTML para el diseño; los datos son ilustrativos. La gráfica muestra la CPU por encima del umbral de 85 % durante los últimos dos minutos, condición que origina la alerta del recuadro inferior derecho. Elaboración propia.

Figura 3

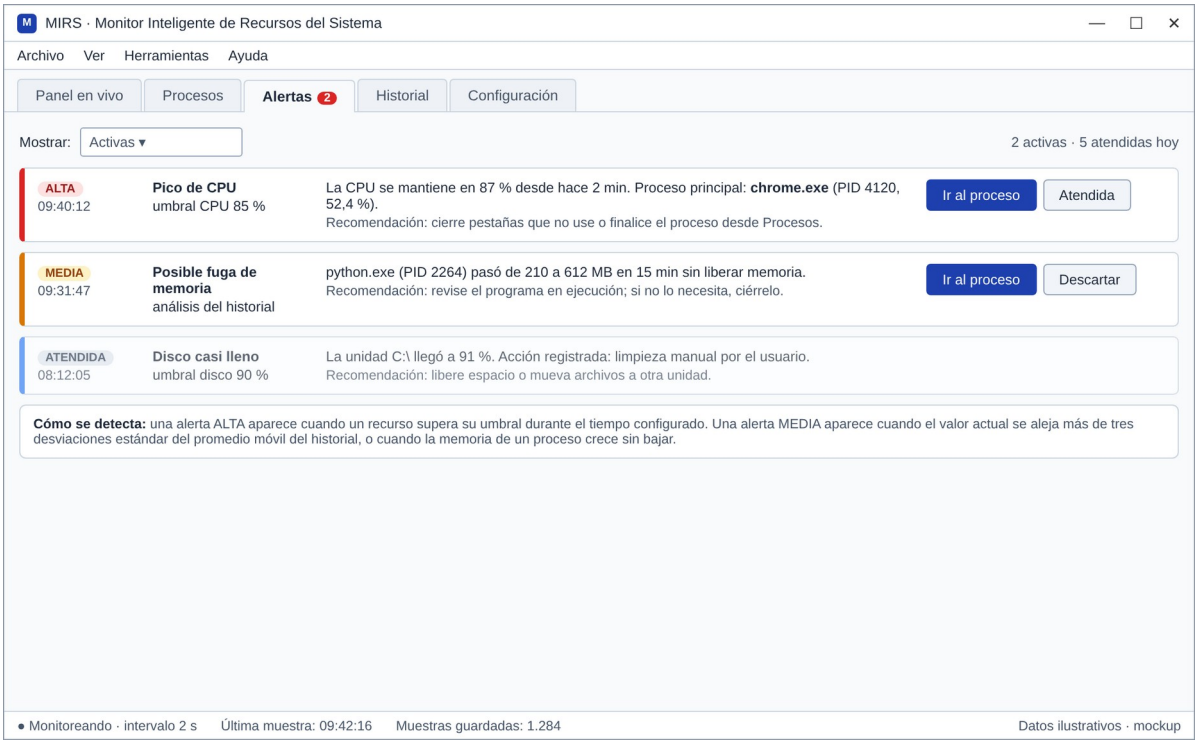
Mockup de la pestaña Procesos con el diálogo de confirmación



Nota. Al pulsar Finalizar proceso se abre un diálogo que indica el consumo del proceso y advierte sobre el trabajo sin guardar. Datos ilustrativos. Elaboración propia.

Figura 4

Mockup de la pestaña Alertas



Nota. Cada alerta indica su severidad con texto y con color, la regla que la originó y una recomendación. Datos ilustrativos. Elaboración propia.

En el diseño se tomaron tres decisiones. Cada estado lleva una etiqueta de texto además del color (Normal, Vigilar, Alerta), para que la información no dependa solo de distinguir colores. La acción de finalizar un proceso aparece en rojo y siempre abre una ventana de confirmación con el botón Cancelar en primer lugar. Por último, la barra de estado informa en todo momento si el monitoreo está activo, cuál es el intervalo y cuándo se tomó la última muestra.

4.3 Control de versiones y commits

El proyecto usa Git, un sistema de control de versiones distribuido en el que cada copia del repositorio conserva la historia completa de cambios (Chacon y Straub, 2014), y GitHub como servidor remoto. Los mensajes de commit siguen la convención Conventional Commits,

que propone el formato tipo(ámbito): descripción para que el historial se pueda leer y procesar con facilidad (Conventional Commits, s.f.). Los tipos usados son feat para funcionalidades, fix para correcciones, docs para documentación, test para pruebas y chore para estructura y configuración. La Tabla 8 presenta los commits de esta entrega en la rama main.

Tabla 8

Commits de la Entrega 2 en la rama main

Hash	Mensaje	Archivos principales
69a64e3	docs: agregar README con la descripción contextual y técnica de MIRS	README.md
8fd2384	chore: crear estructura de carpetas, requirements y .gitignore	Carpetas, dependencias y configuración
b06fae3	feat(bd): definir el esquema SQLite del modelo relacional (6 tablas)	database/schema.sql
fbb6bc1	feat(recolector): leer CPU, RAM, disco, red y procesos con psutil	src/mirs/recolector.py
1695fdb	feat(almacenamiento): guardar equipo, umbrales y muestras en SQLite	almacenamiento.py y __main__.py
63b7b79	test: pruebas unitarias del recolector y del almacenamiento	tests/ (10 pruebas)
2b7bc51	docs: agregar ficha técnica, diagramas de arquitectura y modelo relacional	docs/ficha y docs/diagramas
ea459cb	docs(interfaz): mockups del panel, procesos y alertas	docs/mockups
46ec0a7	fix(recolector): guardar la prioridad como entero (psutil 8 devuelve un enum en Windows)	recolector.py y su prueba

Nota. Todos los commits tienen fecha del 4 de noviembre de 2026. Un commit final agrega este documento a la carpeta docs/. El historial completo puede consultarse en la pestaña Commits del repositorio.

El último commit de la tabla muestra por qué conviene registrar las correcciones por separado. Desde la versión 8, psutil devuelve en Windows la prioridad de un proceso como un miembro de una enumeración de clases de prioridad y no como un número (Rodola, s.f.-b); el recolector la convierte ahora a entero antes de guardarla, y una prueba verifica ese tipo. Para las siguientes entregas cada módulo nuevo se trabajará en una rama feature/<módulo> y se integrará a main cuando sus pruebas pasen.

4.4 Base de datos

El historial se guarda en SQLite, un motor de base de datos SQL que funciona dentro de la misma aplicación, sin servidor ni configuración, y que almacena toda la base en un único archivo (SQLite, s.f.-a). Python lo incluye en su biblioteca estándar a través del módulo sqlite3 (Python Software Foundation, s.f.-a), de modo que el usuario no necesita instalar nada adicional. El archivo datos/mirs.db se crea la primera vez que se ejecuta el prototipo, a partir de database/schema.sql.

Tres decisiones de configuración protegen la calidad de los datos. La primera tiene que ver con las llaves foráneas: SQLite las trae desactivadas por compatibilidad y deben activarse en cada conexión (SQLite, s.f.-b), por lo que el módulo de almacenamiento ejecuta PRAGMA foreign_keys = ON cada vez que se conecta. La segunda es el modo WAL, en el que los lectores no bloquean al escritor ni el escritor a los lectores (SQLite, s.f.-c); gracias a él la interfaz puede consultar el historial mientras el hilo de muestreo escribe. La tercera consiste en guardar cada muestra junto con sus procesos en una sola transacción, de modo que, si una fila falla, no queda una muestra incompleta. La Tabla 9 resume las seis tablas.

Tabla 9

Diccionario de datos resumido

Tabla	Llave primaria	Llaves foráneas	Restricciones principales
equipo	id_equipo	Ninguna	nombre_host único; núcleos, RAM y disco mayores que 0
umbral	id_umbral	id_equipo	recurso en CPU, RAM, DISCO o RED; valor_limite mayor que 0; un umbral por recurso y equipo
muestra	id_muestra	id_equipo	cpu_pct, ram_pct y disco_pct entre 0 y 100; tráfico de red no negativo; índice por equipo y fecha
proceso_muestra	id_proceso_muestra	id_muestra	PID, CPU y memoria no negativos; se borra en cascada con su muestra

Tabla	Llave primaria	Llaves foráneas	Restricciones principales
alerta	id_alerta	id_muestra; id_umbral (admite nulo)	tipo, severidad (BAJA, MEDIA, ALTA) y estado (ACTIVA, ATENDIDA, DESCARTADA) restringidos
accion	id_accion	id_equipo; id_alerta (admite nulo)	tipo FINALIZAR o CAMBIAR_PRIORIDAD; confirmada 0 o 1; resultado restringido

Nota. El esquema completo, con tipos y valores permitidos, está en database/schema.sql del repositorio.

Las reglas de la base de datos se comprobaron con pruebas automáticas. La Figura 5 muestra la ejecución de las diez pruebas: siete verifican el almacenamiento (creación de las seis tablas, equipo sin duplicados, un umbral por recurso, rechazo de una llave foránea inexistente, rechazo de un porcentaje de 150 % y una transacción fallida que no deja procesos huérfanos) y tres verifican el recolector. Al ejecutar después python -m mirs --muestras 5, la base quedó con un equipo, tres umbrales, cinco muestras y cincuenta registros de procesos, diez por cada muestra.

Figura 5

Ejecución de las pruebas unitarias del prototipo

```
$ python -m unittest discover -s tests -v
test_check_rechaza_porcentaje_fuera_de_rango ... ok
test_crea_las_seis_tablas ... ok
test_guardar_muestra_con_sus_procesos ... ok
test_llave_foranea_rechaza_equipo_inexistente ... ok
test_muestra_fallida_no_deja_procesos_huerfanos ... ok
test_registrar_equipo_dos_veces_no_duplica ... ok
test_umbrales_por_defecto_uno_por_recurso ... ok
test_datos_equipo_completos ... ok
test_listar_procesos_respeta_el_tope_y_el_orden ... ok
test_muestra_en_rangos_validos ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.476s

OK
```

Nota. Salida del comando python -m unittest discover -s tests -v sobre el código del repositorio. Elaboración propia.

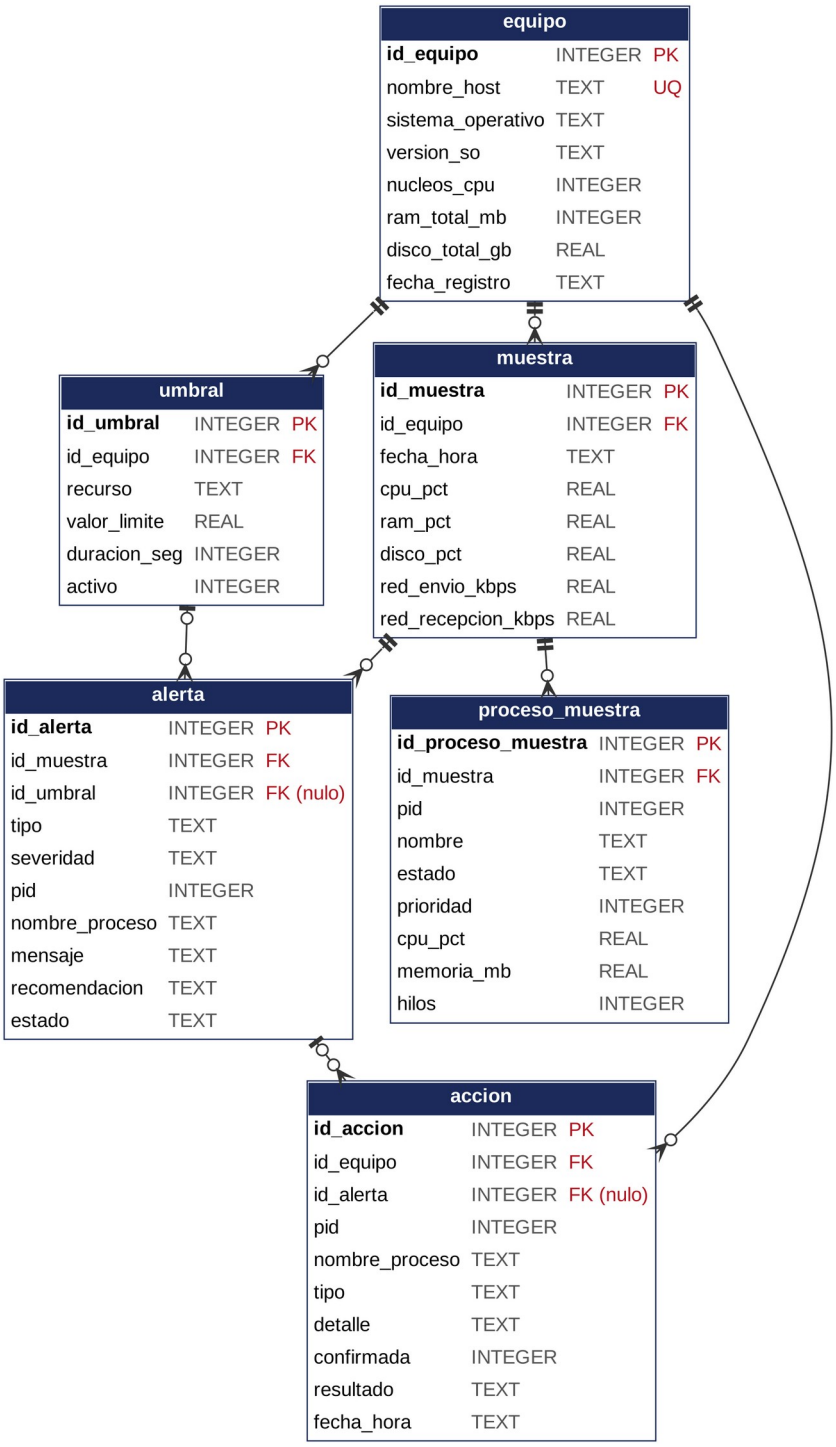
4.5 Modelo relacional

El modelo relacional organiza los datos en relaciones independientes unidas por llaves, siguiendo la propuesta de Codd (1970). La Figura 6 presenta las seis tablas con notación de pata de gallo. Las cardinalidades son las siguientes y coinciden con las llaves foráneas del esquema SQL:

1. Un equipo tiene cero o muchas muestras, cero o muchos umbrales y cero o muchas acciones; cada muestra, umbral o acción pertenece a exactamente un equipo.
2. Una muestra tiene cero o muchos registros en proceso_muestra y origina cero o muchas alertas; cada uno de ellos pertenece a exactamente una muestra.
3. Un umbral origina cero o muchas alertas; una alerta se relaciona con cero o un umbral, porque las anomalías detectadas por el historial no dependen de un umbral fijo.
4. Una alerta origina cero o muchas acciones; una acción se relaciona con cero o una alerta, porque el usuario también puede actuar desde la lista de procesos.

Figura 6

Modelo relacional de la base de datos de MIRS



Nota. PK: llave primaria; FK: llave foránea; UQ: valor único. Dos rayas indican exactamente uno; círculo y raya, cero o uno; círculo y pata de gallo, cero o muchos. Elaboración propia.

Una decisión de diseño merece explicación. No existe una tabla de procesos permanente porque un PID identifica a un proceso solo mientras este existe; cuando termina, el sistema operativo puede asignar el mismo número a otro programa. Por eso proceso_muestra guarda una fotografía de los procesos de mayor consumo en cada muestra, y el análisis de crecimiento de memoria compara el mismo PID y nombre en muestras consecutivas. Los datos del equipo se registran una sola vez en su tabla y las demás los referencian por llave, de manera que ningún atributo se repite fuera de la relación a la que pertenece.

4.6 Lo que queda definido y lo que sigue

Al cierre de esta entrega quedan definidos la arquitectura, las pantallas, el modelo de datos con su esquema SQL, la convención de commits y las reglas de detección, y funcionan con pruebas el recolector y el almacenamiento. La Entrega 3 implementará el planificador, el motor de análisis y las alertas; la Entrega 4, la interfaz Tkinter y el gestor de procesos, y la Entrega 5 cerrará con las pruebas de los requerimientos no funcionales y el manual de usuario.

5. Tecnologías del prototipo

Las tecnologías se eligieron con tres criterios: que permitieran leer el sistema operativo sin depender de herramientas externas, que no exigieran instalar servidores en el equipo del usuario y que el grupo pudiera dominarlas durante el semestre. La Tabla 10 las presenta con su uso en el proyecto.

Tabla 10

Tecnologías seleccionadas

Tecnología	Versión	Uso en MIRS	Justificación
Python	3.10 o superior	Lenguaje de todo el prototipo	Multiplataforma y con bibliotecas maduras para consultar el sistema operativo
psutil	5.9 o superior	Lectura de CPU, memoria, disco, red y procesos	Una sola interfaz para Windows, Linux y otros sistemas (Rodola, s.f.-a)
Tkinter (ttk)	Incluida en Python	Interfaz gráfica de escritorio	No requiere instalación adicional y ofrece controles con apariencia nativa
Matplotlib	3.8 o superior	Gráficas de tendencia en la interfaz	Biblioteca de gráficas 2D para Python que se integra en ventanas Tk (Hunter, 2007)
SQLite (sqlite3)	Biblioteca estándar	Historial local	Sin servidor y en un solo archivo (SQLite, s.f.-a)
threading	Biblioteca estándar	Hilo de muestreo	Separa la recolección de la interfaz para que la ventana no se congele
unittest	Biblioteca estándar	Pruebas automáticas	Viene con Python y basta para el alcance del proyecto
Git y GitHub	Git 2.x	Control de versiones y repositorio remoto	Historial completo, ramas y trabajo en equipo (Chacon y Straub, 2014)
Visual Studio Code	Actual	Editor de código	Gratuito, con soporte para Python y Git

psutil resuelve además un detalle técnico propio de los sistemas operativos. El porcentaje de CPU de un proceso se calcula entre dos lecturas: la primera llamada devuelve un 0,0 que debe descartarse, y el valor puede superar el 100 % cuando el proceso ejecuta varios hilos en procesadores distintos (Rodola, s.f.-b). El recolector toma por eso una lectura base, espera medio segundo, vuelve a leer y divide el resultado por el número de núcleos lógicos, de forma que el valor que ve el usuario queda siempre entre 0 y 100 %.

Se descartó programar directamente contra las interfaces propias de Windows porque habrían atado el código a un solo sistema. Con psutil el mismo recolector funciona en Linux, lo que permitió ejecutar las pruebas fuera de Windows y deja abierta una versión para ese sistema en el futuro.

6. Conclusiones

El levantamiento mostró que el usuario común de Windows sí tiene a su alcance información sobre el consumo de su equipo, pero solo del instante presente, y que las herramientas con historial y alertas están pensadas para administradores. MIRS ocupa ese espacio: conserva el historial, lo compara con el comportamiento habitual del propio equipo y traduce los datos del sistema operativo en alertas con una recomendación. Con los requerimientos, las reglas de detección y el alcance definidos, el proyecto tiene límites claros y verificables.

La estructura presentada deja el prototipo listo para crecer por módulos. La arquitectura en capas aísla la interfaz del muestreo; el modelo relacional, con sus restricciones y transacciones, protege la calidad del historial, y el repositorio registra cada paso con commits descriptivos. Que el recolector y el almacenamiento ya funcionen con pruebas reduce el riesgo de las entregas siguientes, centradas en el análisis, las alertas y la interfaz.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Chandola, V., Banerjee, A., y Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Artículo 15. https://doi.org/10.1145/1541880.1541882

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/es/v1.0.0/

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Microsoft. (s.f.). *Use Performance Monitor to identify performance problems* [Módulo de formación]. Microsoft Learn. https://learn.microsoft.com/en-us/training/modules/monitor-windows-server-performance/2-use-performance-monitor-to-identify-performance-problems

Python Software Foundation. (s.f.-a). *sqlite3: DB-API 2.0 interface for SQLite databases*. Python documentation. https://docs.python.org/3/library/sqlite3.html

Python Software Foundation. (s.f.-b). *tkinter: Python interface to Tcl/Tk*. Python documentation. https://docs.python.org/3/library/tkinter.html

Rodola, G. (s.f.-a). *psutil: Process and system utilities for Python*. https://psutil.io/

Rodola, G. (s.f.-b). *API reference: psutil*. https://psutil.io/api/

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

SQLite. (s.f.-a). *About SQLite*. https://www.sqlite.org/about.html

SQLite. (s.f.-b). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

SQLite. (s.f.-c). *Write-ahead logging*. https://www.sqlite.org/wal.html

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.). Pearson.